



BURGESS ENG.HR.SEC. SCHOOL BILASPUR(C.G.)



SESSION: - 2025-2026

SCHOOL AFFILIATION NO. - 3330043

Computer Science

TOPIC :- Database system using
python and mysql



SUBMITTED BY:

**NAME- Dheeraj Kumar
Sahu & Ishita Pandey**

CLASS - 'XII' 'A'

SUBMITTED TO: -

**Mr.Manish Rajput
(C.S Teacher)**

Certificate

This is to certify that Dheeraj Kumar Sahu & Ishita Pandey of class 'XII' 'A' of **BURGESS ENGLISH
HR. SEC. SCHOOL BILASPUR (C.G.)**

Has successfully completed the project on the topic: -
Database using Mysql & python under the guidance of
Mr. Manish Rajput (Computer science teacher)
during the academic year 2025-2026 in the practical
fulfillment of the project.

Date

Internal's signature

Principal's signature

External's signature

Acknowledgement

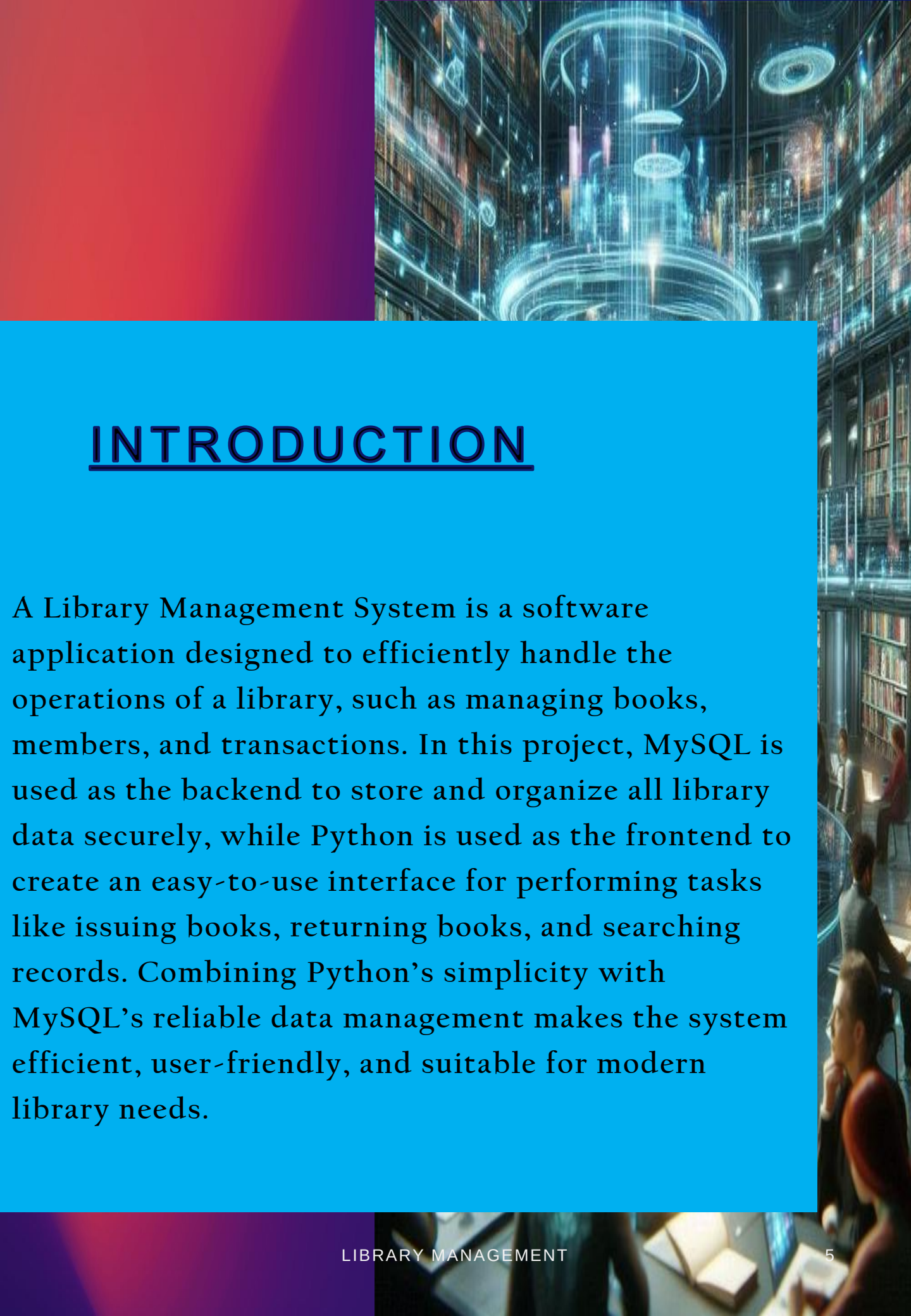
I would like to express my special thanks to my **Mr. Manish Rajput** (Computer science teacher) for their able guidance and support in completing my project. I would also like to extend my gratitude and support in completing my project on the topic: Database using mysql & python

I would also like to extend my gratitude to the principal Madam “Mrs. Nishita Hansa Das” for providing me with all the facility that was required

LIBRARY MANAGEMENT



www.skolaro.com



INTRODUCTION

A Library Management System is a software application designed to efficiently handle the operations of a library, such as managing books, members, and transactions. In this project, MySQL is used as the backend to store and organize all library data securely, while Python is used as the frontend to create an easy-to-use interface for performing tasks like issuing books, returning books, and searching records. Combining Python's simplicity with MySQL's reliable data management makes the system efficient, user-friendly, and suitable for modern library needs.

PROBLEM

Problems in the Old Library Reception System

Manual record-keeping

- **Librarians had to write book and member details in registers, which was time-consuming and prone to mistakes.**

Difficulty in searching records

- **Finding information about books, availability, or member history required checking multiple registers manually.**

Risk of data loss or damage

- **Paper records could be easily lost, torn, or damaged, leading to permanent loss of important information.**

Inaccurate book tracking

- **It was hard to track which books were issued, returned, or overdue, leading to confusion and mismanagement.**

No automatic fine calculation

- **Fines for late returns had to be calculated manually, which often caused errors.**

Limited reporting and analysis

- **Generating reports like the number of issued books, overdue books, or member activity was difficult without computerized support.**

Slow service at reception

- **Because all tasks were manual, there were long queues and delays during issuing or returning books.**

SOLUTION

How the Library Management System Solves These Problems

Digital record-keeping with MySQL

- All data is stored securely in a structured database, reducing errors and making management easier.

Quick search and retrieval

- Python-based search features allow librarians to find any book or member record instantly.

Data backup and security

- MySQL provides secure storage and backup options, preventing data loss.

Automated book tracking

- The system automatically updates issued, returned, and available books in real time.

Auto fine calculation

- The system calculates late fines automatically based on predefined rules.

Efficient reporting

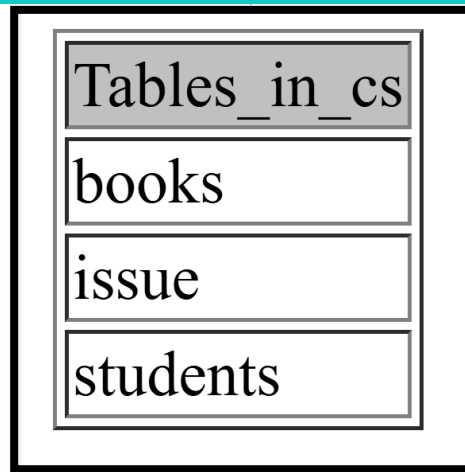
- The system can generate reports like daily transactions, overdue lists, or stock summaries with a single click.

Faster service

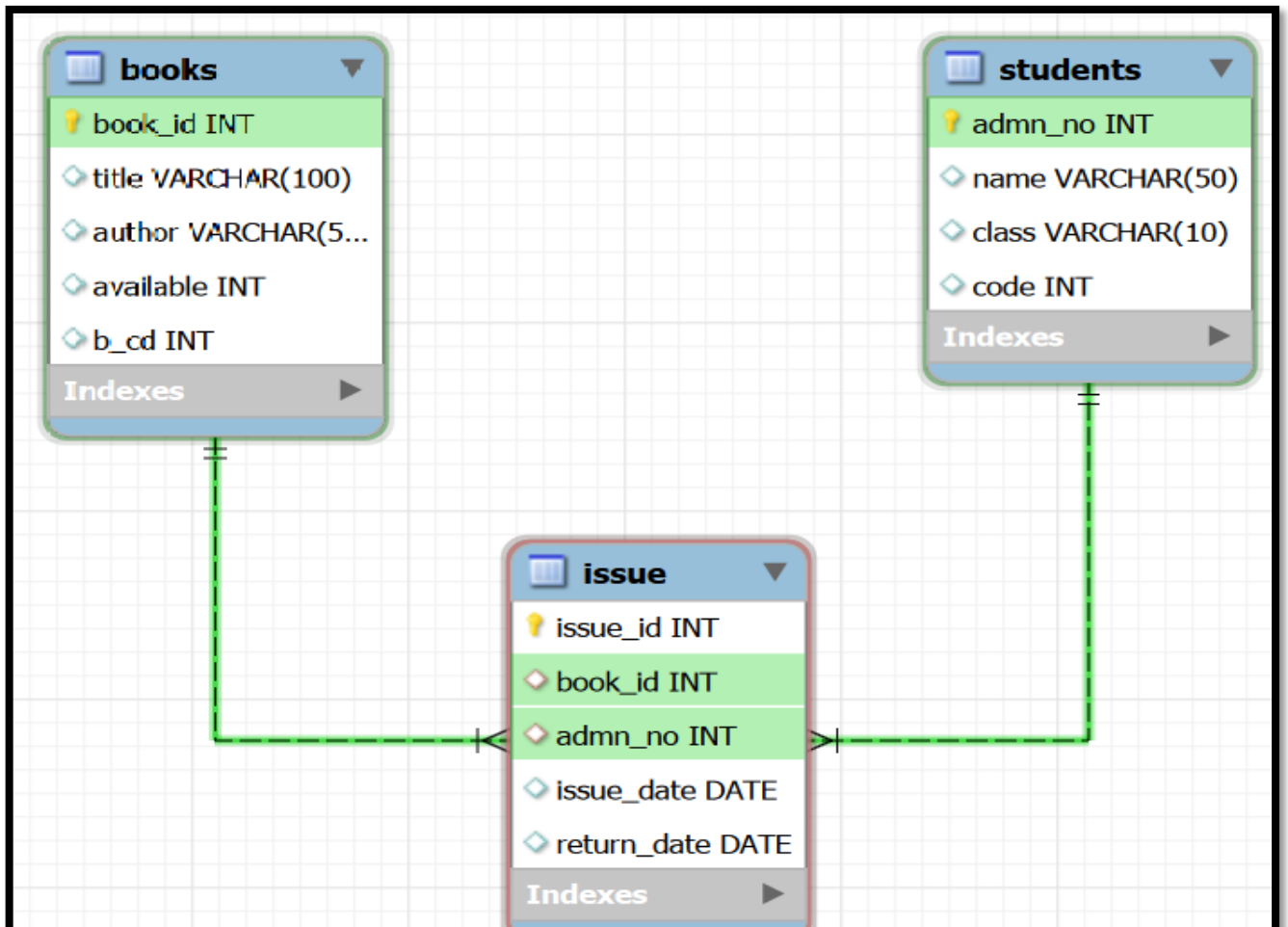
- Automated processes reduce workload and speed up issuing, returning, and registration at the reception.

PRODUCT OVERVIEW

DATABASE (BACKEND)

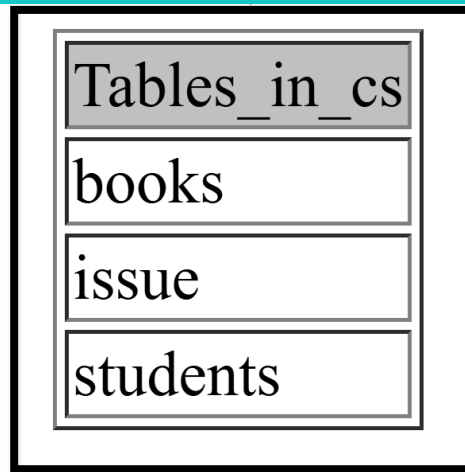


Connection Between Tables

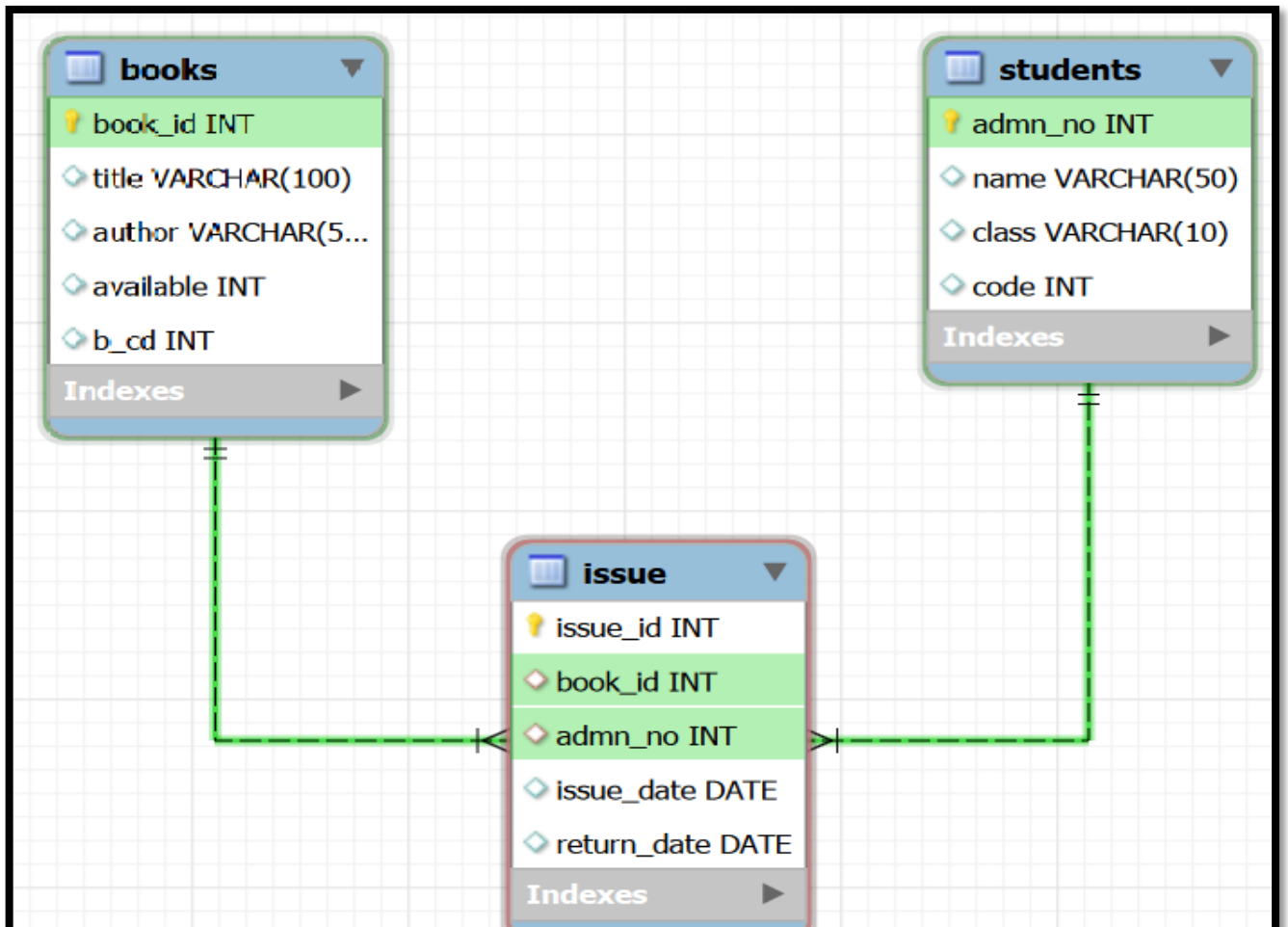


PRODUCT OVERVIEW

DATABASE (BACKEND)



Connection Between Tables



Structure of Student Table

Field	Type	Null	Key	Default	Extra
admn_no	int	NO	PRI	NULL	
name	varchar(50)	YES		NULL	
class	varchar(10)	YES		NULL	
code	int	YES		NULL	

Structure of Book Table

Field	Type	Null	Key	Default	Extra
book_id	int	NO	PRI	NULL	auto_increment
title	varchar(100)	YES		NULL	
author	varchar(50)	YES		NULL	
available	int	YES		NULL	
b_cd	int	YES		NULL	

Structure of Issue Table

Field	Type	Null	Key	Default	Extra
issue_id	int	NO	PRI	NULL	auto_increment
book_id	int	YES	MUL	NULL	
admn_no	int	YES	MUL	NULL	
issue_date	date	YES		NULL	
return_date	date	YES		NULL	

```

import mysql.connector as sql
from datetime import date

# Connect to MySQL
con = sql.connect(host="localhost", user="root",
password="dheeraj@19", database="cs")
cur = con.cursor()

def admin_login():
    print('Hello Admin Enter your password ! ->')
    P_w=input('Enter Password :- ')
    if P_w == 'dk@19':
        print("Hello User Access Granted ☑ ")
        def book_m ():
            def add_book1():
                title = input("Enter book title: ")
                author = input("Enter author name: ")
                avail =input("Enter availibility(Quantity) of
book :- ")
                bkcd=int(input("Enter Book Code :- "))
                cur.execute("INSERT INTO books (title, author,
available,b_cd) VALUES (%s, %s, %s,%s)", (title,
author,avail,bkcd))
                con.commit()
                print("☑ Book added successfully!")

            def view_books1():
                print("##### List of all Books in this
Library #####")
                cur.execute("SELECT * FROM books order by
book_id")
                for row in cur.fetchall():
                    print(row)
            def del_book1():
                print("////////// Drop Book
//////////")

```

```

        cur.execute("SELECT title,b_cd FROM books order
by book_id")
        for row in cur.fetchall():
            print(row)
        title1= input("Enter the Book-code to Delete it
:- ")
        query='''delete from books where b_cd =%s'''
        val=(title1,)
        cur.execute(query,val)
        con.commit()
        print(" ! Book Deleted successfully !")

def v_i():
    cur.execute("select * from issue as _1 left
join students as _2 on _1.admn_no=_2.admn_no where return_date
is null ;")
    for l__ in cur.fetchall():
        print("=====Book issued=====")
        print("Name      :    ",l__[6])
        print("Class    :    ",l__[7])
        print("book_id:    ",l__[1])
        print("Admn_no:    ",l__[2])
        print("Issue_id:    ",l__[0])
        print("Issue_date: ",l__[3])
        print("return_date:",l__[4])

def update_bka():
    print("##### List of all Books in this
Library #####")
    cur.execute("SELECT
book_id,title,available,b_cd FROM books order by book_id")
    for row in cur.fetchall():
        print(row)
        bid=int(input("Select the Book to Update
avalibility :- "))
        ava=int(input('Update avalibility of the Book
:- '))

```

```

        query=("update books set available =%s WHERE
book_id =%s ")
        val=(ava,bid)
        cur.execute(query,val)
        con.commit()
        print("Avalibility updated")

def menu():
    while True:
        print("\n===== BOOK MENU =====")
        print("1. Add Books")
        print("2. View Books")
        print("3. Delete Book")
        print("4. View Issued Books")
        print("5. Update avalibility")
        print("6. Exit")

        choice = input("Enter choice: ")
        if choice == '1':
            add_book1()
        elif choice == '2':
            view_books1()
        elif choice == '3':
            del_book1()
        elif choice == '4':
            v_i()
        elif choice == '5':
            update_bka()
        elif choice == '6':
            print("👋 Exiting...")
            break
        else:
            print("👋 Invalid choice! Try again.")

    menu()
def stu_m():

```



```

def add_student1():
    admn_no = int(input("Enter Admission No: "))
    name = input("Enter Student Name: ")
    clas = input("Enter Class: ")
    code1= int(input("Enter Student-Code: "))
    cur.execute("INSERT INTO students VALUES (%s,
%s, %s,%s)", (admn_no, name, clas,code1))
    con.commit()
    print("☑ Student added successfully!")

def view_student1():
    print("##### List of all Registered Students
in this Library #####")
    cur.execute("SELECT * FROM students ORDER BY
admn_no")
    for stu in cur.fetchall():
        print(stu)

def delete_student1():
    print("//////////      Drop IDs
//////////")
    cur.execute("SELECT name,code FROM students
order by admn_no")
    for row in cur.fetchall():
        print(row)
    title2= input("Enter the code to Delete it :-
")
    query1=''delete from students where code
=%s''
    val=(title2,)
    cur.execute(query1,val)
    con.commit()
    print(" ! Student ID Deleted successfully !")

def menu():
    while True:
        print("\n*****Student Menu *****")

```

```

        print("1. Add Student")
        print("2. View Student ")
        print("3. Delete Student")
        print("4. Exit")

        choice = input("Enter choice: ")
        if choice == '1':
            add_student1()
        elif choice == '2':
            view_student1()
        elif choice == '3':
            delete_student1()
        elif choice == '4':
            print("\n Exiting...")
            break
        else:
            print("\n Invalid choice! Try again.")

    menu()
def v_i():
    cur.execute("select * from issue as _1 left join students as _2 on _1.admn_no=_2.admn_no where return_date is null ;")

    print("issue_id,book_id,date of issue,(DOR),Name,Student_id")
    for vissue in cur.fetchall():
        print(vissue)

def menu():
    while True:
        print("\n*****Admin Access *****")
        print("1. Book menu")
        print("2. Student Menu")
        print("3. View Issued Book")
        print("4. Exit")

```

```

        choice = input("Enter choice: ")
        if choice == '1':
            book_m()
        elif choice == '2':
            stu_m()
        elif choice == '3':
            v_i()
        elif choice == '4':
            print("\n Exiting...")
            break
        else:
            print("\n Invalid choice! Try again.")

    menu()
else:
    print('Asscess Denied Wrong Password !!!!')

def Bk_menu():
    def view_books():
        print("##### List of all Books in this Library
#####")
        cur.execute("SELECT book_id,title,author, available
FROM books order by book_id")
        for row in cur.fetchall():
            print(row)

    def issue_book():
        print("====DISPLAYING ONLY AVAILABLE BOOKS =====")
        cur.execute("SELECT book_id,title,author FROM books
WHERE available > 0")
        for b in cur.fetchall():
            print(b)
        book_id = int(input("Enter Book ID From The Above list:
"))
        admn_no = int(input("Enter Admission No: "))
        cur.execute("SELECT available FROM books WHERE
book_id=%s", (book_id,))

```



```

        avail = cur.fetchone()
        if avail and avail[0]:
            cur.execute("INSERT INTO issue (book_id, admn_no,
issue_date, return_date) VALUES (%s, %s, %s, NULL)", (book_id,
admn_no, date.today()))
            cur.execute("UPDATE books SET available= available-
1 WHERE book_id=%s", (book_id,))
            con.commit()
            cur.execute("SELECT * FROM issue ORDER BY issue_id
DESC LIMIT 1")
            for k in cur.fetchall():
                print (k)
                print("📖 Book issued successfully!")
                print(f'Your _Issued_ID_={k[0]} for 📖 Book:-
{book_id}')
                print(f'Please Remember you Issued_ID :- {k[0]} At
the time of Returning Book')

        else:
            print("📖 Book not available!")

def return_book():
    issue_id = int(input("Enter Issue ID: "))
    cur.execute("SELECT book_id FROM issue WHERE
issue_id=%s AND return_date IS NULL", (issue_id,))
    data = cur.fetchone()
    if data:
        cur.execute("UPDATE issue SET return_date=%s WHERE
issue_id=%s", (date.today(), issue_id))
        cur.execute("UPDATE books SET available= available
+1 WHERE book_id=%s", (data[0],))
        con.commit()
        print("📖 Book returned successfully!")
    else:
        print("📖 Invalid Issue ID or already returned!")
    cur.execute("SELECT * FROM issue WHERE admn_no =
%s", (issue_id,))

```

```

def view_fines():
    days= int(input("Enter number of days Books Borrowed :-
"))
    if (days<=5):
        amt1=0.50 * days
        print (f"Fine Amount to be paid :-{amt1} ")
    elif (days <= 10):
        amt2=1 * days
        print (f"Fine Amount to be paid :-{amt2} ")
    elif (days >10 and days<20 ):
        amt3=5 * days
        print (f"Fine Amount to be paid :-{amt3} ")
    elif(days >=20):
        amt4=10*days
        print(f'Fine Amount to be paid :-{amt4} \nYour
Registration Could be Canceled')
def menu():
    while True:
        print("\n##### BOOK MENU #####")
        print("1. View Books")
        print("2. Issue Book")
        print("3. Return Book")
        print("4. View Fine ")
        print("5. Exit")

        choice = input("Enter choice: ")
        if choice == '1':
            view_books()
        elif choice == '2':
            issue_book()
        elif choice == '3':
            return_book()
        elif choice == '4':
            view_fines()
        elif choice == '5':
            print("👋 Exiting...")

```



```

            break
        else:
            print("❌ Invalid choice! Try again.")
    menu()
def reg_stu():
    print("##### List of all Registered Students\n#####")
    cur.execute("SELECT admn_no,name,class FROM students\nORDER BY admn_no")
    for stu in cur.fetchall():
        print(stu)

def cus_login():
    cs1=int(input("Enter admn_no :- "))
    query1="SELECT * FROM students WHERE admn_no =%s"
    val=(cs1,)
    cur.execute(query1,val)
    for l in cur.fetchall():
        print("*****Details*****")
        print("admn_no: ",l[0])
        print("Name: ",l[1])
        print("Class: ",l[2])
        print("code: ",l[3])
    query2="SELECT * FROM issue WHERE admn_no = %s"
    val1=(cs1,)
    cur.execute(query2,val1)
    for l_ in cur.fetchall() :
        print("=====Book issued=====")
        print("issue_id: ",l_[0])
        print("book_id: ",l_[1])
        print("admn_no: ",l_[2])
        print("Issue_date: ",l_[3])
        print("return_date: ",l_[4])
        if (l_[3] == l_[4]):
            print(" No Late Fee ")
        else:

```

```

        print("*****Late Fee will be charged***** \n
Please find you Fee by another Menu")

    else:
        print("=====No More Books issued=====")

def menu():
    while True:
        print("\n===== LIBRARY MANAGEMENT MENU
=====")
        print("1. Admin login")
        print("2. Books Menu")
        print("3. List of Regitered Student [ID]")
        print("4. Customer Login(issued Books and Details)")
        print("5. Exit")

        choice = input("Enter choice: ")
        if choice == '1':
            admin_login()
        elif choice == '2':
            Bk_menu()
        elif choice == '3':
            reg_stu()
        elif choice == '4':
            cus_login()
        elif choice == '5':
            print("👋 Exiting...")
            break
        else:
            print("👋 Invalid choice! Try again.")

menu()
con.close()

```



PRODUCT BENEFITS

Simple and efficient to use

Quick customer service
assistance

SUMMARY

Library Management System Using MySQL and Python

The Library Management System is a software application designed to streamline and automate the daily operations of a library. The system uses MySQL as the backend database and Python as the frontend interface. MySQL is responsible for securely storing and managing all library data, including books, members, issued books, returns, fines, and staff details. Its relational structure ensures efficient data retrieval, integrity, and scalability.

Python serves as the frontend and application logic layer. Through Python's user-friendly interface—often built using libraries such as Tkinter, PyQt, or a command-line interface—the system allows librarians to easily perform tasks such as adding new books, registering members, issuing and returning books, searching records, and generating reports. The connection between Python and MySQL is handled through connectors like `mysql-connector-python` or `PyMySQL`, enabling smooth communication and real-time data updates.

Overall, the system improves efficiency, reduces manual effort, minimizes errors, and offers a reliable, organized way to manage library operations. The combination of Python's simplicity and MySQL's robustness makes the application flexible, secure, and easy to maintain.

THANK YOU

MADE BY :-

DHEERAJ SAHU

ISHITA PANDEY

DHEERAJKUMARSAHU827@GMAIL.COM

